Course: Individual Software Development Process' 2026

Software Proposal

By BigNiceMilk (BNM)

Chosen Irl Challenge: Project C: Department Information & Communication Hub

Suwat Teiwasuculmetri	6810545956
Phunnipath Theankaew	6810545816
Apichai Pattanakamolkul	6810545972
Chayapol Kaewsakul	6810545557
Suppamok Tosranon	6810545921

Table of Contents

1. Target Users

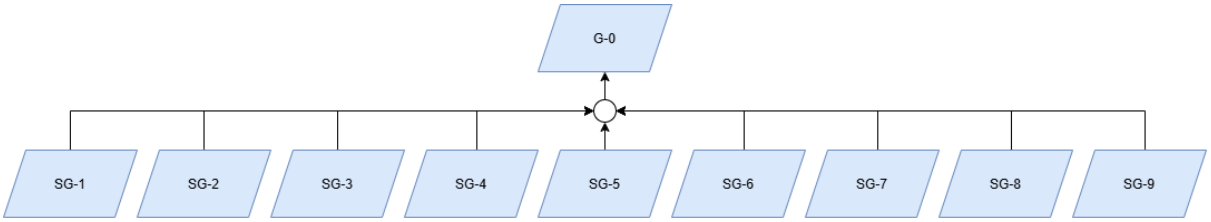
- **CPE & SKE Students (User Level)**: Need a centralized, noise-free space to quickly find year-specific academic updates, lab/professor directories, and course deadlines.
- **Department Admins & Lecturers (Admin Level)** : Need a streamlined, low-overhead interface to publish official announcements once without cross-posting across multiple third-party apps.

USER CONTEXT
Guest / Non-logged in: Read-only access strictly to public department announcements and public FAQs.
Student: View course posts, join via class code, create/join groups with live capacity, submit Q&A forum threads, view personalized DoomScroll feed.
Lecturer / TA: Quick Post CRUD for assigned courses, create class codes, manage course rosters, answer/pin Q&A threads, post reel media.
Department Admin: Global override, user role assignment, system-wide moderation.

2. The System Goal

G-0: Gather all information and announcements in one place.
Establish a centralized, easily navigable, and simple-to-maintain digital hub that unifies academic information, operational guidance, and official updates, eliminating redundant staff inquiries and providing students with a single source of truth for course and department logistics.

3. Goal Refinement via KAOS:



- **Sub-Goal 1 (SG-1)**
  Description: Achieve : Unify scattered announcements and information into a single platform.
  Rationale: Eliminates the need for students to check multiple platforms (e.g., LINE, Discord, Google Classroom), saving them time when searching for updates.
- **Sub-Goal 2 (SG-2)**
  Description: Achieve: Reduce effort for posting official announcement

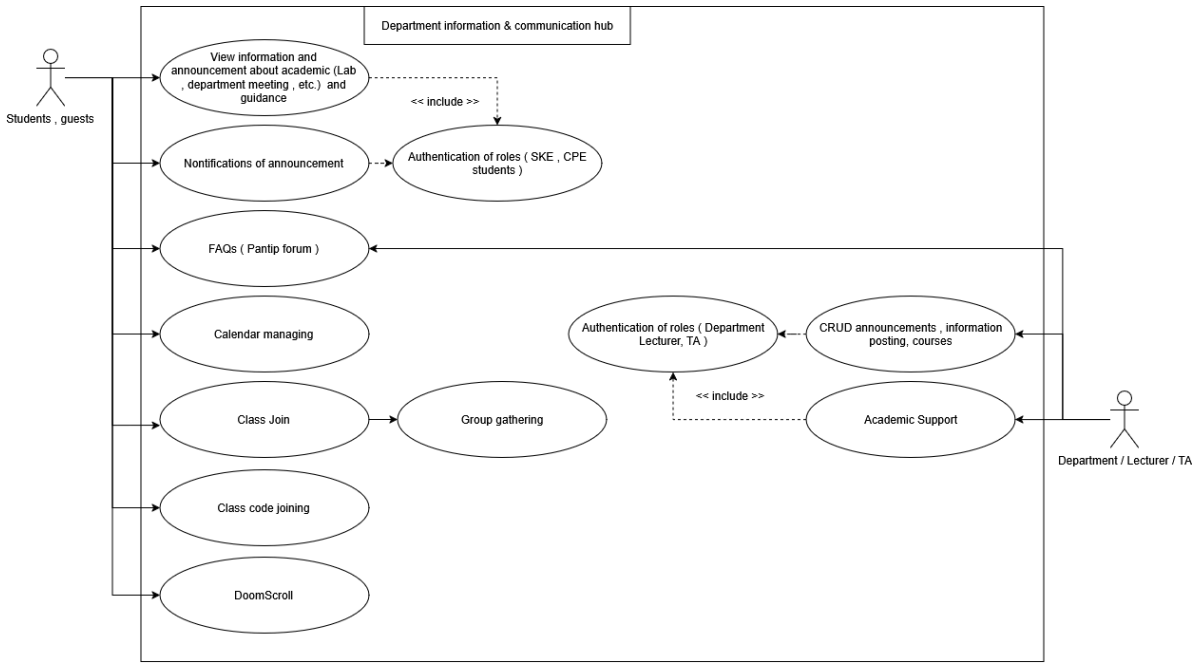
Rationale: Admin level can post updates directly through a Quick Post UI without crossposting across multiple external platforms, significantly reducing administrative time.

- **Sub-Goal 3 (SG-3)**
  Description: Maintain : Schedule and Deadline Synchronization
  Rationale: Having calendars, notifications, deadline date, and digest emails. Maintaining students or noticing Lecturers and students to stay on-time and accountable.
- **Sub-Goal 4 (SG-4)**
  Description: Avoid : Avoiding information mismatch
  Rationale: Enforcing role-based target audiences for posts prevents students from receiving irrelevant updates, eliminating confusion and information noise.
- **Sub-Goal 5 (SG-5)**
  Description: Achieve **:** Google OAuth2 credentials.
  Rationale: Restrict site authentication exclusively to validated `@ku.th` Google OAuth2 credentials. Securely accessible only to verified Kasetsart University personnel.
- **Sub-Goal 6 (SG-6)**
  Description: Maintain : Provide global full-text search capabilities across all posted news, lab topics, and historical announcements.
  Rationale: Guarantees instant retrieval of past information, preventing repeat inquiries to department staff.
- **Sub-Goal 7 (SG-7)**
  Description: Maintain: Self-Service Problem Resolution & Academic Support.
  Rationale: Provide a centralized, Pantip-style Q&A forum and FAQ repository where students can search for frequently asked questions, access verified answers, and directly seek academic support when self-service resources are insufficient. This approach helps students resolve common problems efficiently while reducing repetitive support workload for staff and administrators.
- **Sub-Goal 8 (SG-8)**
  Description: Achieve: Automated Teammate, Project Matchmaking, and Joining class with code.
  Rationale: Facilitate student team formation for courses. Eliminates informal, scattered team recruitment across external chat groups, ensuring every student can find compatible project partners within CPE/SKE. Able students to join class with unique class code to make joining class easier for students.

- **Sub-Goal 9 (SG-9)**
  Description: Maintain : Doom scroll studying

Rationale: New generation students love to doom-scroll instagram reels or tiktok short all the time. Maintaining them to doom-scroll but about what they're studying instead will make them gain more knowledge.

4. Use Case Diagram



5. User Stories

User Story ID	Description	Acceptance Criteria
US-01	As a system user, I want to log in using my Google @ku.th account, so that I can securely access department and course resources.	The system authenticates users via Google OAuth2. Access is granted only to emails ending in @ku.th; all non-@ku.th login attempts are rejected.
US-02	As a non-logged-in user, I want to view academic announcements and information made by the department, so that I can stay informed on general updates.	The public announcement board strictly displays official, global department-level announcements. Course-specific and internal lab posts are hidden from unauthenticated users.
US-03	As a student, I want to search for accurate and precise information about courses from Lecturers or TAs in one place, so that I don't have to look across multiple platforms.	The platform provides a global query search bar with full-text search capability, allowing students to filter up-to-date posts by keyword, course code, or tag.
US-04	As a Lecturer or TA, I want to create, update, and delete course information and syllabus materials for my responsible courses, so that I can keep content accurate with minimal effort.	The system provides a Quick Post UI with explicit actions to create, edit, or delete posts. Lecturers can only manage content for courses assigned to them.

US-05	As the Department Admin, I want to assign/edit user roles and manage all global announcements or lower-level lectures, so that overall system integrity is maintained.	Department Admins have global access to create, edit, or delete any post (including those created by Lecturers) and can update user permission levels across the system.
US-06	As a student, I want to get notified only for important department announcements and enrolled courses, so that I am not spammed with irrelevant updates.	Logged-in users with specific tags (e.g., SKE, CPE, 0110532) receive automated email/in-app notifications when an authorized user publishes a post with the "Notify" box checked.
US-07	As a student, I want an easy way to join a course without using long course IDs, so that enrollment is fast and accurate.	A "Join Class" modal is visible on the UI, allowing students to enter a unique class code generated by the lecturer to gain instant course access.
US-08	As a Lecturer, I want an easy way to create a class and add students via student IDs or generated class codes, so that roster management is simple.	The creation UI allows lecturers to set up a class, generate a unique 6-character class code, and manually add student IDs under the "Members" tab.
US-09	As a non-logged-in user, I want to view frequently asked questions made in the department, so that I can find answers to common inquiries without signing in.	The public FAQ section displays general department Q&A posts in read-only mode for unauthenticated users.

US-10	As a student, I want to create questions and read FAQs for my specific department or enrolled courses, so that I can resolve study doubts easily.	The interactive FAQ section provides a post-and-reply feature scoped to specific courses and departments for authenticated users.
US-11	As a lecturer or TA, I want to provide academic guidance and answer student questions on my course, so that students receive official support.	Course TAs and Lecturers can review, answer, and pin verified responses on course-specific Q&A threads created by students.
US-12	As a student, I want an integrated calendar to track my assignments and announcements, so that I never miss submission dates or events.	The system aggregates homework submission deadlines and scheduled announcement dates into a synchronized visual calendar for enrolled courses.
US-13	As a student, I want an easy way to find and join a group, see open slots, and view members in real-time when assigned group work, so that team formation is seamless.	The group list clearly displays current capacity (e.g., 3/4), updates member rosters in real time, and is restricted exclusively to students enrolled in that specific course.
US-14	As a student, I want a bite-sized video/card feed of lecture materials, so that I can review course content quickly without reading through long files.	The system provides a mobile-friendly "DoomScroll" reel feed that algorithmically suggests short, tagged study snippets matching the student's enrolled courses.

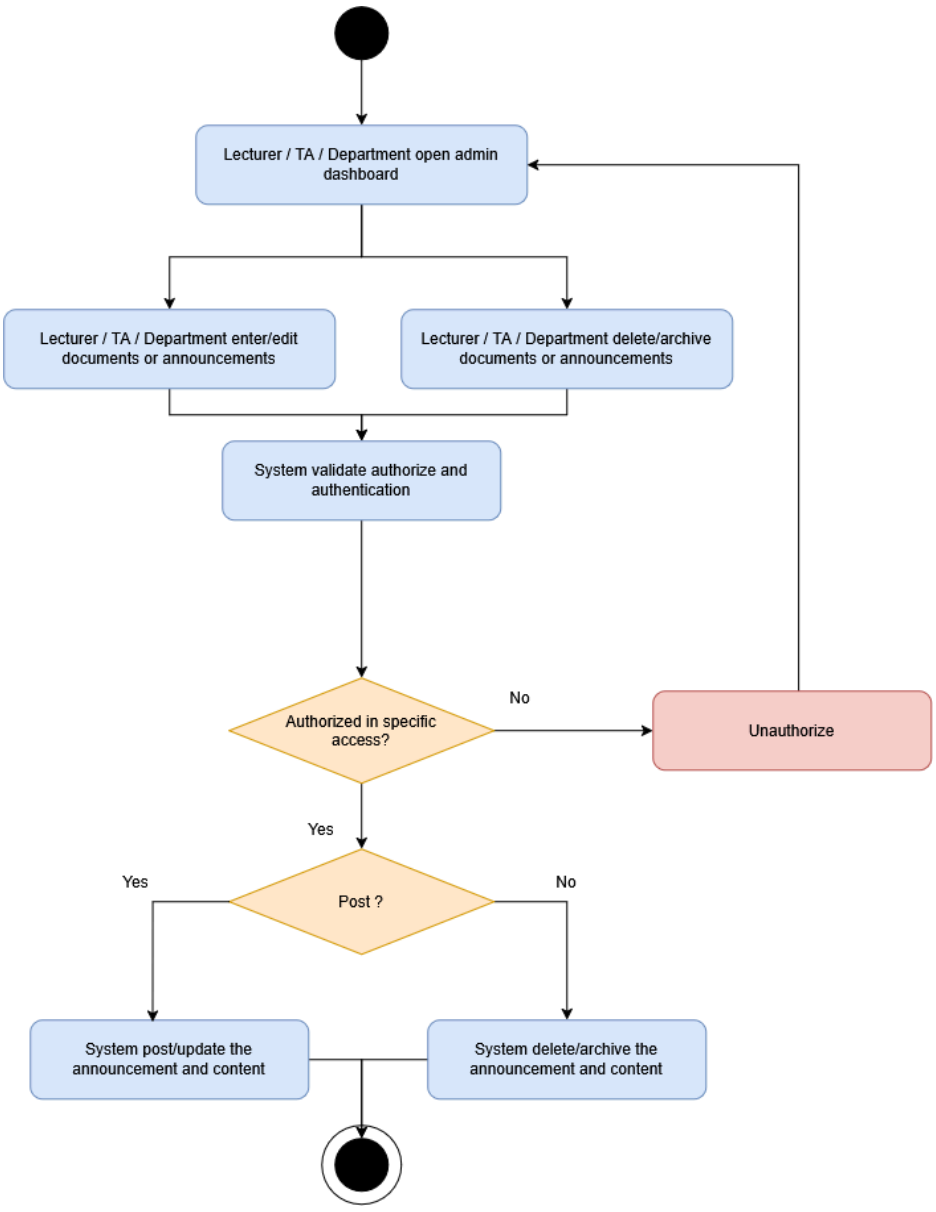
6. User Requirement Specification

ID	User Requirement
URS-1	The system shall restrict user access and authentication strictly to validated `@ku.th` Google OAuth2 credentials.
URS-2	Students shall be able to view information in one hub and keep track of notification and calendar.
URS-3	Users can do full-text query search across all active and archived announcements, course materials, and lab details.
URS-4	Students can ask questions on the dashboard (FAQs) and be able to reach out for support from the department or their course.
URS-5	Students shall be able to view all the groups for working to see which one is full and which one they can join. Then they should be able to request to join the group. and manage real-time group work formation with visible member lists and slot capacity limits in each course.
URS-6	Students must be able to join course classes via unique class codes and be able to view course's work.
URS-7	Admin level (Department / Lecturer / TA) must be able to upload / edit / delete their information about their corresponding subject and announcements.
URS-8	Admin level (Department / Lecturer / TA) must be able to give out support within their subject / department.
URS-9	The department must be able to post a video for the DoomScroll feature and students must be able to interact with it.

7. Activity Diagram

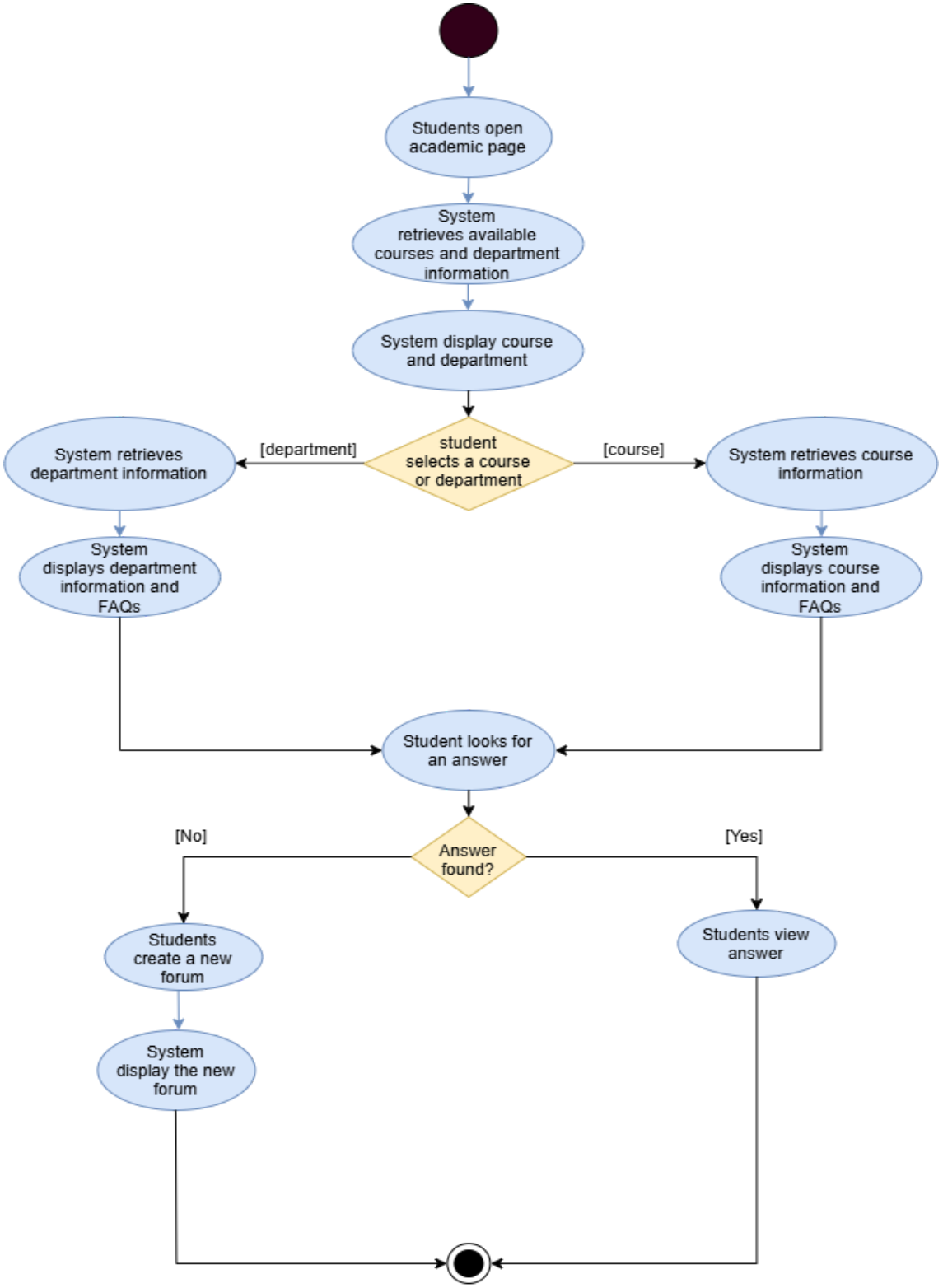
AD-1: Department / Lecturer / TA can create, update or edit documents or information.

This diagram was used during elicitation to discover system responsibilities, decisions, and exception paths, which were then converted into SRS-9 below.



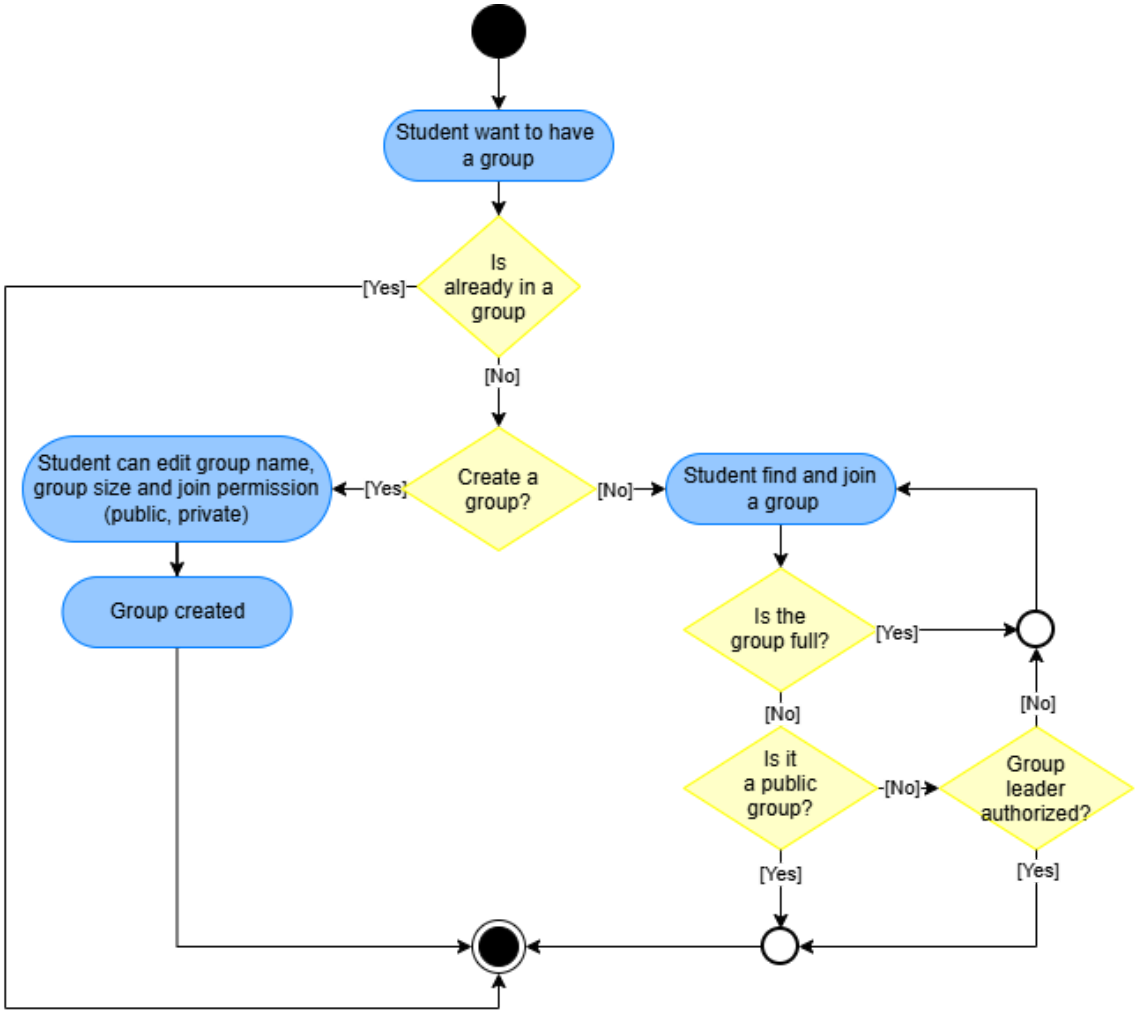
AD-2: Student Views information about academic, guidance, and FAQ

This diagram shows how students can view academic information, guidance and FAQ from the department and for each course they are in separately. Which then connected to SRS-3 below.



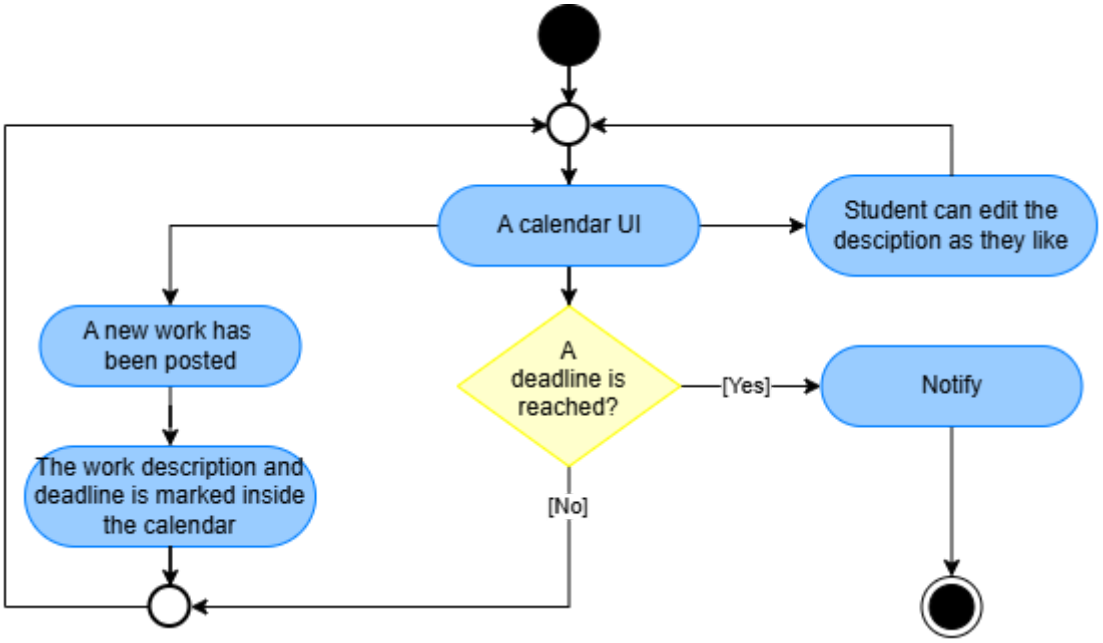
AD-3: Student can join group in each class easily

This diagram captures the authorisation check and availability-update flow used to derive SRS-7.

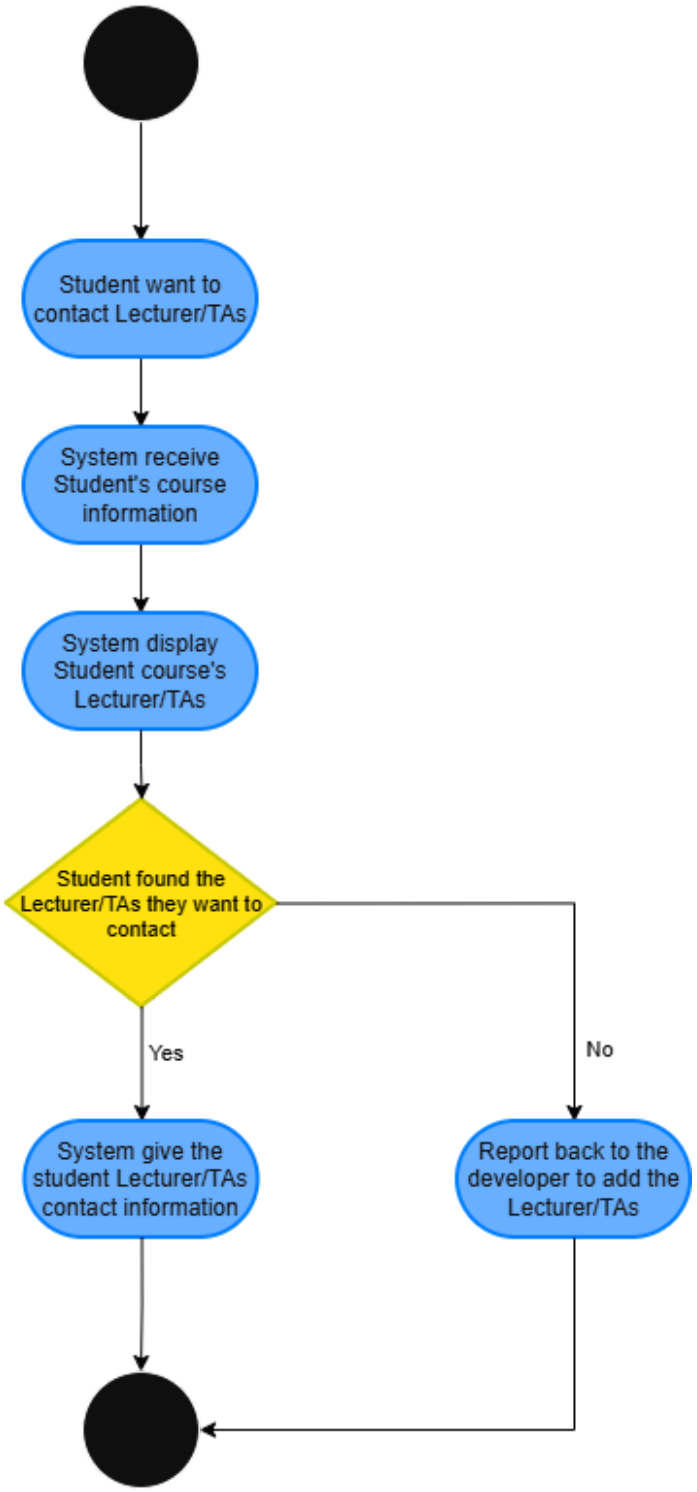


AD-4: Lecturers, TAs, and students can manage their own calendar. Be able to receive notifications on important announcements or information.

This diagram shows how lecturers, TAs, and students can manage their own calendar, edit their deadline or any works, to be able to receive notifications. And also receive notification on any information or announcements. This can be converted to SRS-6 and SRS-4.

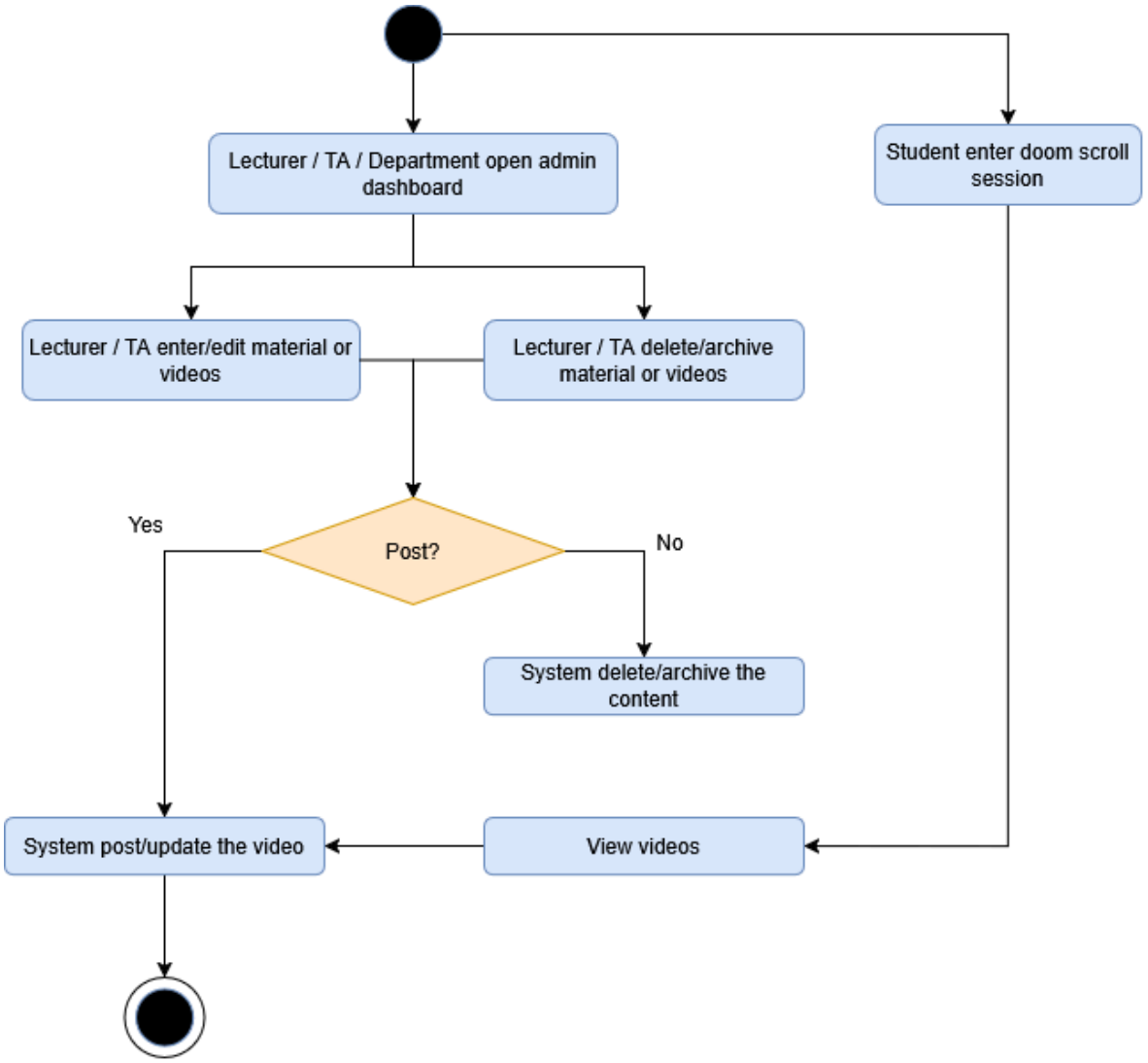


AD-5: Students can contact administrators, lecturers, or TAs if they are willing to.
If students have an important question they can directly contact an administrator, lecturers, and TAs via contact information. This can be converted to SRS-10.



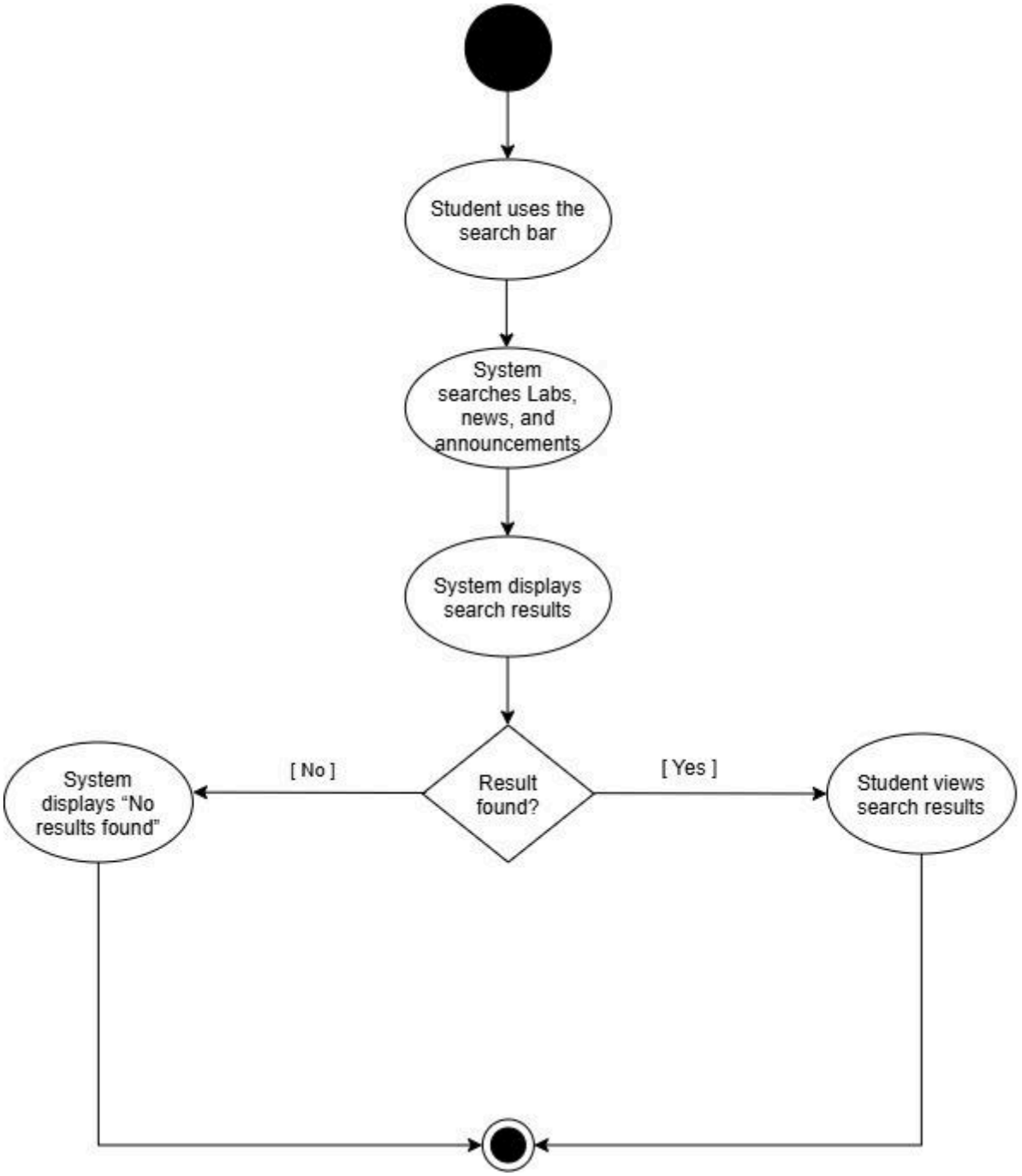
AD-6: Student can scroll short videos for academic study

Similar to Tiktok, Instagram reels, facebook reels, and youtube shorts. Students can scroll through short videos about academics, courses they're taking or department announcement short videos. All about academic information. Converting into SRS-8.



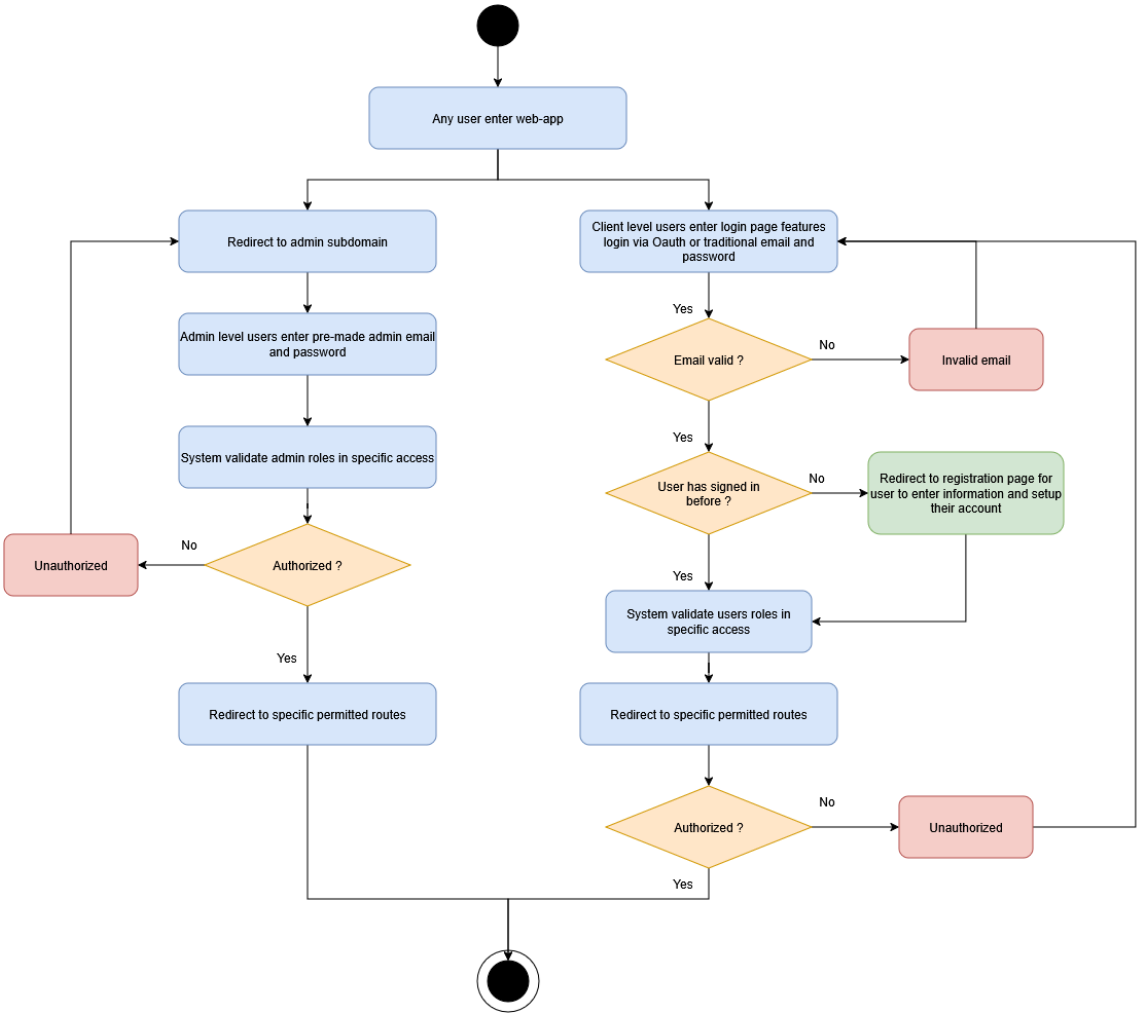
AD-7: Students can search for accurate and precise information about courses.

This diagram shows how students can use the search engine to find accurate and precise information about courses from Lecturers, TAs, and the Department without having to search across multiple platforms. Students can enter keywords into the search bar and receive relevant and up-to-date course information, announcements, and other academic information. This can be converted to SRS-1



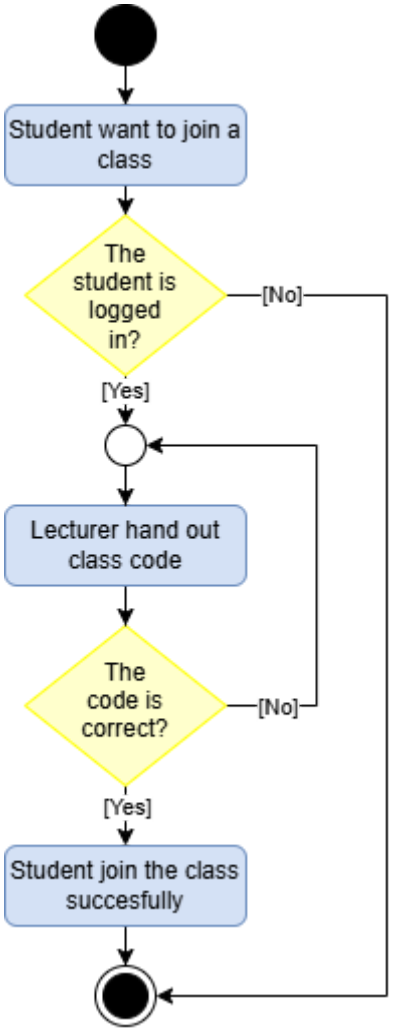
AD-8 : Authentication and authorization of roles and permissions of each users (admin / client)

This diagram shows how authentication and authorization works for adding a role with permissions for each user. By detecting a specific email which is "@ku.th" using google oauth2 to identify the role the user must have, giving user permission exclusively for each role. Identifying if the user is an admin or a client. This can be converted to SRS-2.



AD-9 : Student join a class with class code

This diagram shows how the class joining system works, this page can only be seen after the student is logged in to their account. This can be converted to SRS-11.



8. System Requirement Specification

ID	System Requirement
SRS-1	The system shall feature an indexed, full-text search engine allowing users to query announcements, laboratory details, news posts, and historical materials by keyword or tag.
SRS-2	The system must use Google oauth2 authentication that only accept @ku.th type and identify user roles (Department, Lecturer , TA, Student's major)
SRS-3	The system shall provide a centralized dashboard displaying global department announcements, course updates, and syllabus information.
SRS-4	The system shall notify students, lecturers and TAs to be on-time and accountable. Notifying them about deadlines or any important announcements.
SRS-5	The system shall display frequently asked questions on the main department page. Also on each course page, students shall be able to create a new forum if they can't find the answer to their question.
SRS-6	The system shall be able to let students, lecturers, and TAs to manage their own calendar.
SRS-7	The system shall be able to let students create their own group work for each course group work. Allowing them to manually request or accept to be a member of the group.
SRS-8	The system shall be able to let students scroll through short videos about lectures or courses they're taking. The scrolling should be looping and never end.
SRS-9	The system shall allow only authorised administrators to create, edit, change availability, or archive academic information and documents. The video will either be made by lecturers or TAs or Professional academic employees
SRS-10	The system shall be able to let students contact an administrator, or any academic support if they are willing to.
SRS-11	The system shall let students join courses with code from authorized access from the lecturer.

9. Software Architecture

Based on the SRS for the Department Information & Communication Hub (Iteration 1 scope: centralized information, **Q&A/FAQ**, group matching, doomscroll feed, calendar, and role-based management), the selected architecture is an **Event-Driven Modular Monolithic Architecture** using the Model-View-Controller **(MVC)** with Service Layer pattern. This is a single deployable application, internally organized into clearly separated domain modules and layered components, rather than a flat monolith or a fully distributed microservices system.

Rationale

The system is being built to consolidate scattered academic announcements, course materials, team matchmaking, and video content into a single platform for CPE & SKE students and department staff. A full microservices split would introduce unnecessary operational overhead (service discovery, inter-service networking, distributed transactions, and multi-repo deployment pipeline management) that would slow down development without immediate necessity, as there is no requirement in the SRS for independent scaling of micro-components. Conversely, a flat, undifferentiated monolith risks becoming tightly coupled and unmaintainable as complex domain rules grow—such as Google OAuth2 domain validation (@ku.th), role-based access control (Admin, Lecturer, TA, Student), full-text search indexing, real-time group slot tracking, calendar deadline sync, and asynchronous notification dispatch.

The Modular Monolithic approach with an Event-Driven component provides the deployment simplicity of a single application unit while strictly encapsulating business logic into isolated modules (Auth, Content/Search, Group Gathering, Calendar/Notifications, Media). Presentation (View), request routing/authorisation (Controller), domain logic (Model/Service), and async background processing (Event Engine) remain cleanly separated. This structure keeps the codebase maintainable across team members and ensures that high-load modules (such as the DoomScroll media streaming service or Search indexer) can be cleanly extracted into independent microservices in future iterations if traffic demands increase.

Component	Layer/Type	Description
Web/Mobile Client	Presentation (View)	Renders the public announcement hub, student/lecturer dashboards, Pantip-style Q&A forum, group matching UI, calendar, and DoomScroll reel viewer; consumes API

		data and displays system state, notifications, and validation errors.
Core API Controllers	Controller	Receives HTTP/REST requests across modules (posts, class codes, group join requests, Q&A threads, calendar events, media feeds), delegates authorization checks (SRS-2, SRS-9), and coordinates calls to the domain models before returning responses to the View.
Domain Models & Service Layer	Model	Encapsulates core business rules: validates @ku.th OAuth domains (SRS-2), enforces group slot capacity limits (SRS-7), processes full-text search queries (SRS-1), and handles Q&A thread tagging and pinning logic (SRS-5).
Event Engine & Background Worker	Event-Driven / Async	Listens to system events (e.g., assignment created, announcement checked "Notify") to asynchronously queue and send email/in-app notifications and process video transcribing/caching for the DoomScroll feed without blocking HTTP response cycles (SRS-4, SRS-8).

Main Relational Database	Persistence	Stores structured system records (users, roles, course rosters, announcement posts, Q&A threads, calendar deadlines, and group memberships) (SRS-2, SRS-3, SRS-6, SRS-7) and supports full-text search indexing (SRS-1).
Media & Blob Storage	Storage / CDN	Stores short video files and media assets uploaded for the DoomScroll feature (SRS-8), serving video content efficiently via a Content Delivery Network.
Authentication & RBAC Module	Cross-cutting	Restricts access exclusively to validated @ku.th Google OAuth2 credentials (SRS-2) and enforces role permissions across Admin, Lecturer, TA, and Student access points (SRS-9) prior to any controller write operation.

10. Data Storage Technology

The recommended data storage architecture uses a **Relational Database Management System (RDBMS)** (**MySQL**) as the primary data store, supplemented by a self-hosted, S3-compatible **Object Storage service** (**MinIO**) and an **In-Memory Cache Engine** (**Redis**).

Rationale

The core data elements described in the SRS—users, roles, course rosters, Q&A threads, group member slots, calendar deadlines, and announcements—are highly structured, interconnected, and rely on well-defined relationships. An RDBMS provides strong ACID compliance, transactional integrity, and key constraints essential for critical rules: Google @ku.th role verification (SRS-2), preventing group over-subscription beyond capacity limits (SRS-7), and handling permissions for editing or deleting academic content (SRS-9). Additionally, modern RDBMS engines like PostgreSQL provide built-in full-text search indexing capabilities, directly fulfilling SRS-1 for querying news, lab details, and announcements without requiring a separate, complex search engine cluster.

However, unlike a single-table menu system, this application introduces two distinct storage demands: media streaming (SRS-8) and real-time concurrency with background tasks (SRS-4, SRS-6). Large binary video files for the continuous "DoomScroll" feed (SRS-8) are not stored directly in the relational database, as doing so inflates database size, slows down backups, and hinders streaming performance. Because the system is deployed entirely on local infrastructure rather than a public cloud, media is stored in a self-hosted, S3-compatible object store—specifically MinIO, deployed alongside the rest of the stack in a dedicated Docker container. This maintains identical separation-of-concerns benefits as a cloud object store—dedicated media storage and streaming-friendly access patterns—while operating with zero external service or internet dependencies. For lighter-weight deployments where running an extra storage service is unneeded, video files are written to the local filesystem with only their paths tracked in the RDBMS, traded off against MinIO's built-in redundancy and bucket-policy features.

Redis runs as a local container to manage fast session tokens, buffer real-time group slot locks, and queue background jobs for deadline notification scheduling (SRS-4, SRS-6)—functioning identically in local deployments since Redis operates self-hosted in all environments.

Component	Data Storage Technology	Description
Core Relational Database	RDBMS (MySQL)	Primary relational store containing normalized tables for users, roles, course rosters, announcements, Q&A threads, group listings, and calendar deadlines (SRS-2, SRS-3, SRS-5, SRS-6, SRS-7, SRS-9). Enforces key integrity, authorization relationships, and powers indexed full-text search queries (SRS-1).
Media Object Storage	Self-hosted Object Storage (MinIO), deployed as a Docker container alongside the app	Stores uploaded short video files and media assets for the DoomScroll study feed (SRS-8) via an S3-compatible API. Keeps large binary files out of the relational database to protect query and backup performance, while providing a bucket-based interface for streaming and future portability to cloud object storage if the system scales beyond a single host.
In-Memory Cache & Message Queue	In-Memory Store (Redis)	Handles ephemeral state data, rapid key-value caching, atomic group-slot reservation locks (SRS-7), and queues background notification dispatching jobs for upcoming calendar deadlines and check-notified posts (SRS-4, SRS-6).

This hybrid data storage approach ensures strict relational integrity and precise querying for core academic functions while maintaining high throughput and low latency for media playback and real-time notifications across the entire platform.

11. Development Environment

The recommended development and deployment environment is a containerized environment using Docker, deployed onto local infrastructure (the team's own machines) rather than cloud-hosted infrastructure or bare-metal deployment.

Rationale
The Department Information & Communication Hub must reliably support multiple user roles, including students, TAs, lecturers, and department administrators, across both web and mobile platforms while maintaining consistent performance.

A containerized environment using Docker packages the system's multiple components—MVC application server, RDBMS, object storage, cache engine, and background notification workers—into isolated and reproducible containers. This allows the development team to maintain the same runtime environment across all development machines and the final deployment environment, reducing configuration inconsistencies and preventing common issues such as differences in external dependencies and service configurations.

Docker Compose is used to manage the entire application stack as a single environment, including the application server, PostgreSQL, MinIO, and Redis. This allows all services to be started, stopped, and rebuilt together using a consistent configuration, making development and deployment easier to manage.

The system is deployed on local infrastructure rather than cloud-hosted infrastructure, keeping the application and its data under departmental control. This reduces recurring infrastructure costs and minimizes dependence on external services and networks. Compared with bare-metal deployment, containerization also simplifies dependency management, system updates, environment replication, and recovery. If the system needs to support a larger number of users in the future, the containerized architecture can also be migrated to a larger deployment environment without requiring major changes to the application.

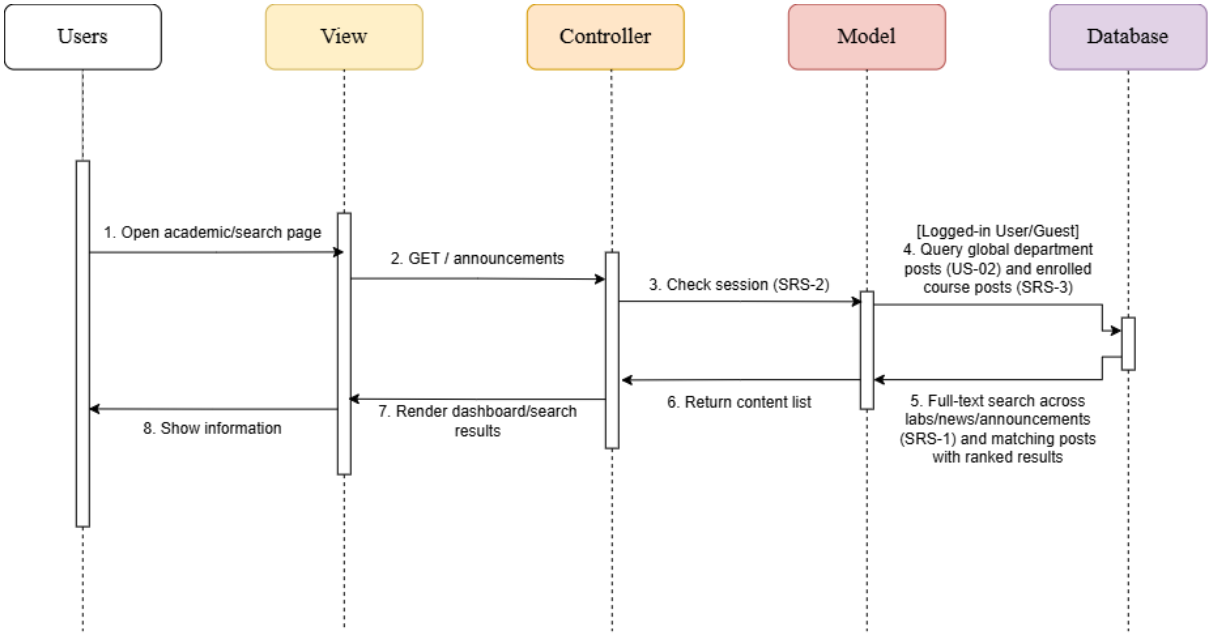
Component	Development Environment	Description
Application Server (Controller + Model)	Docker Container	Packages the MVC application API, OAuth authentication middleware, and business logic layer in a lightweight container, ensuring consistent execution across dev and production environments.
Core Relational Database	Docker Container (via Docker Compose)	Runs the PostgreSQL instance locally for development with pre-configured schema migrations and full-text search extensions enabled, decoupled from host system OS dependencies.
Media Object Storage	Docker Container (via Docker Compose)	Runs a MinIO instance locally, exposing an S3-compatible API for storing and retrieving DoomScroll video files, decoupled from the application server and database.
In-Memory Cache & Task Queue	Docker Container (via Docker Compose)	Runs a Redis container locally to handle rapid key-value caching, atomic lock testing for group slots, and asynchronous event queue processing.
Local Development Engine	Local Docker Desktop / Container Runtime	Allows developers to run the entire multi-service stack locally with a single command (`docker-compose up`), mirroring the live production environment.

12. Sequence Diagrams for All SRS

The following sequence diagrams define the interaction flows for the Department Information & Communication Hub. They illustrate how users and system components interact to fulfill the functional requirements defined in the SRS. Each diagram is traced to its corresponding SRS requirements and architectural decisions (ADs).

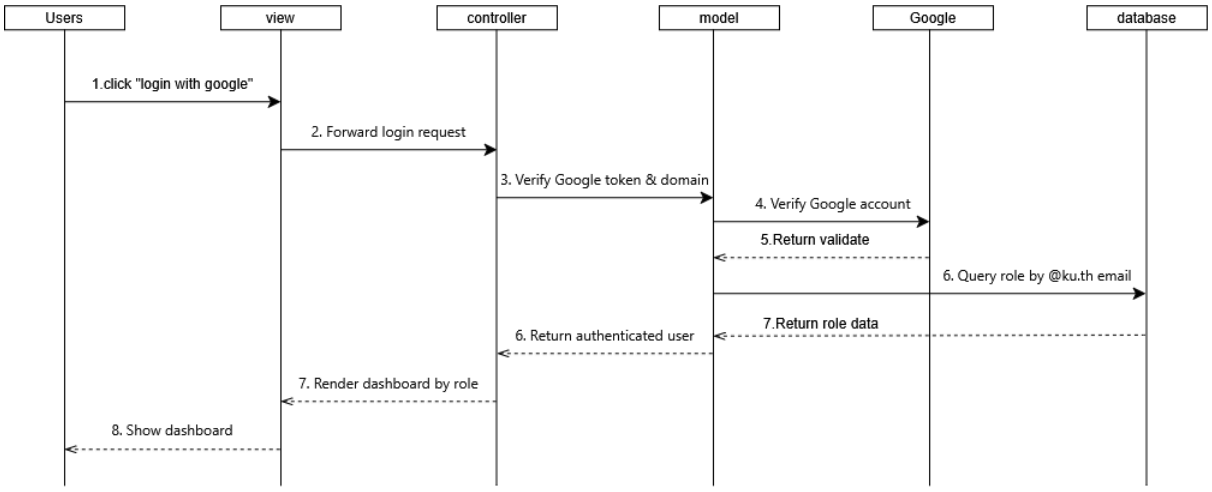
SQD-1: Guest/Student Views Info & Searches

Traced: SRS-1, SRS-2, SRS-3 · from AD-7, AD-2



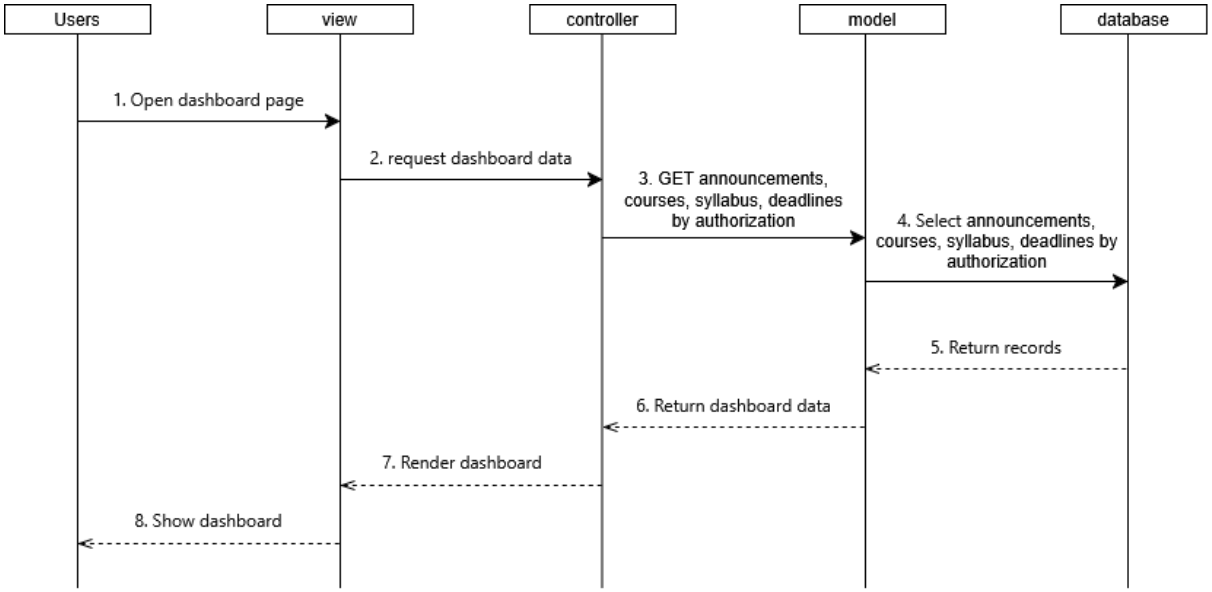
SQD-2: Google oauth2 authentication

Traced requirements: SRS-2



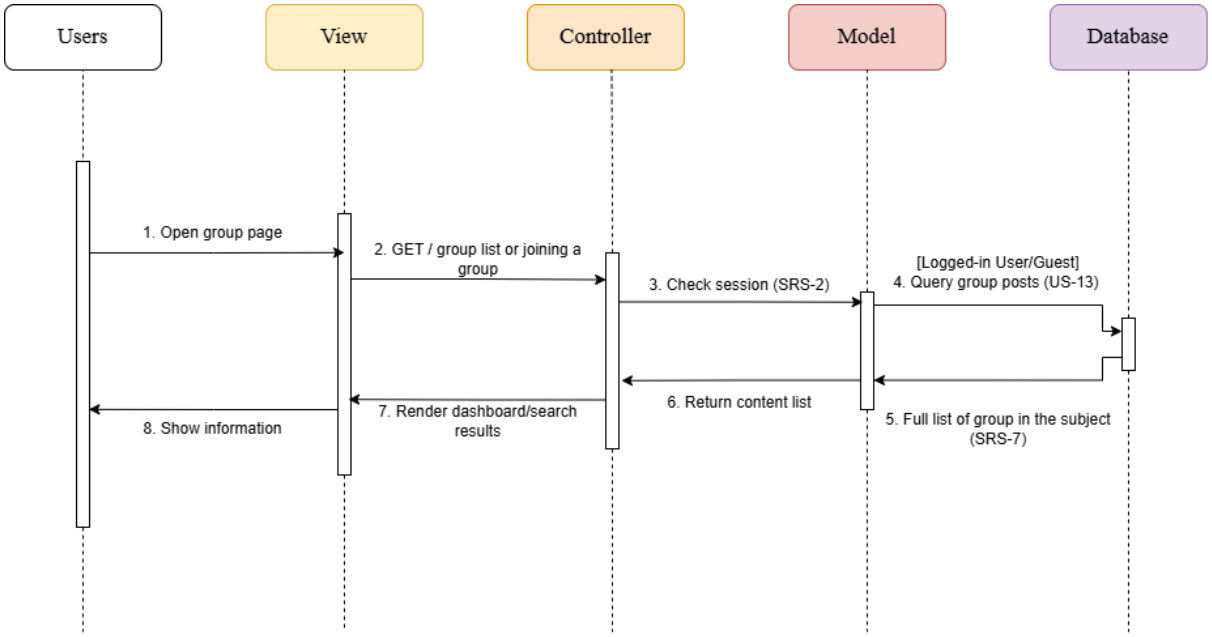
SQD-3: Announcement Dashboard

Traced requirements: SRS-3



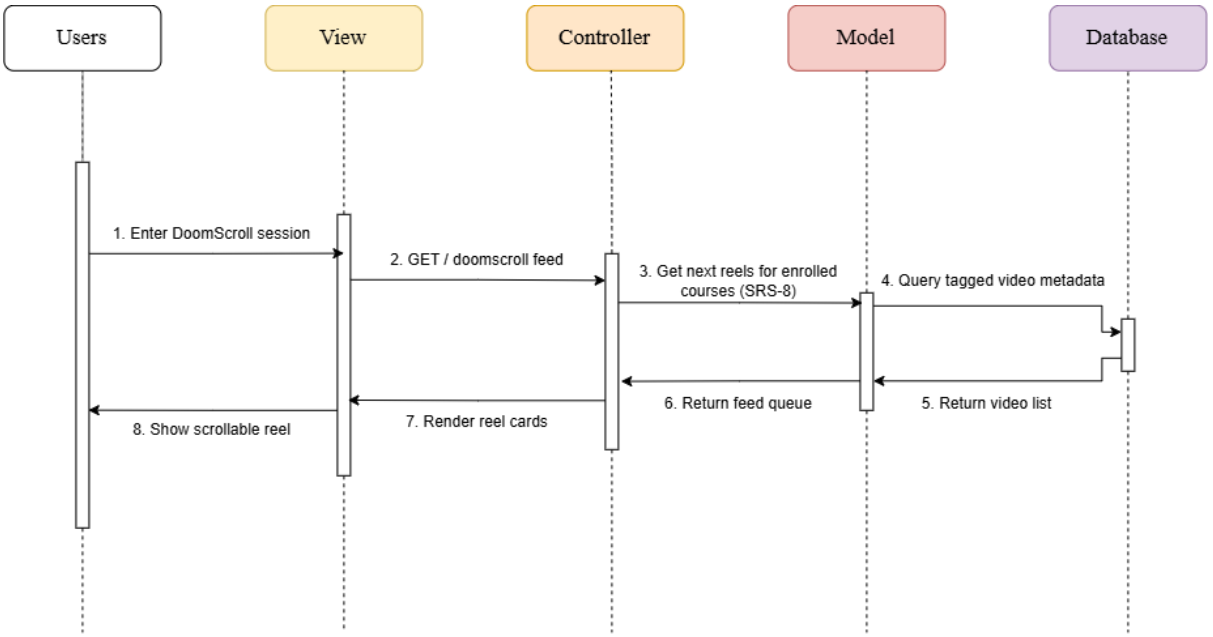
SQD-4: Student Group Gathering

Traced: SRS-7 · from AD-3



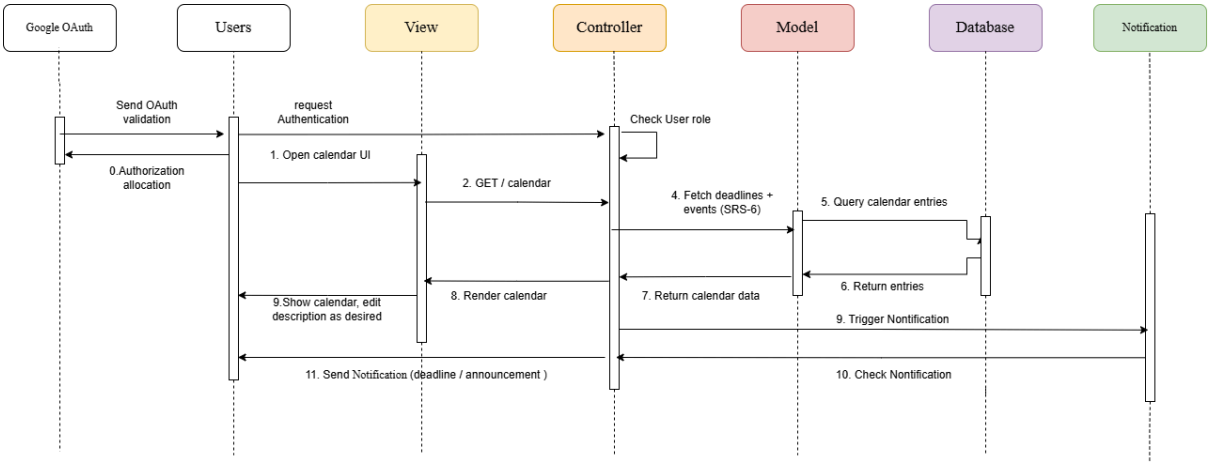
SQD-5: Student Scrolls DoomScroll Feed

Traced: SRS-8 · from AD-6



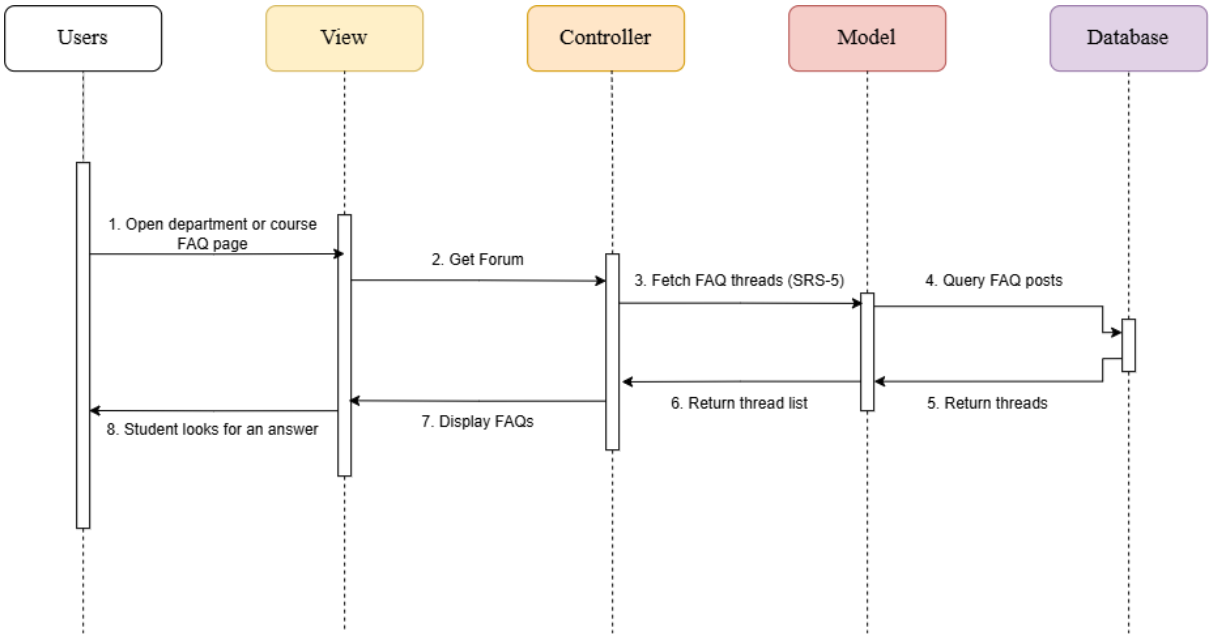
SQD-6: Calendar Management & Notifications

Traced: SRS-4, SRS-6 · from AD-4



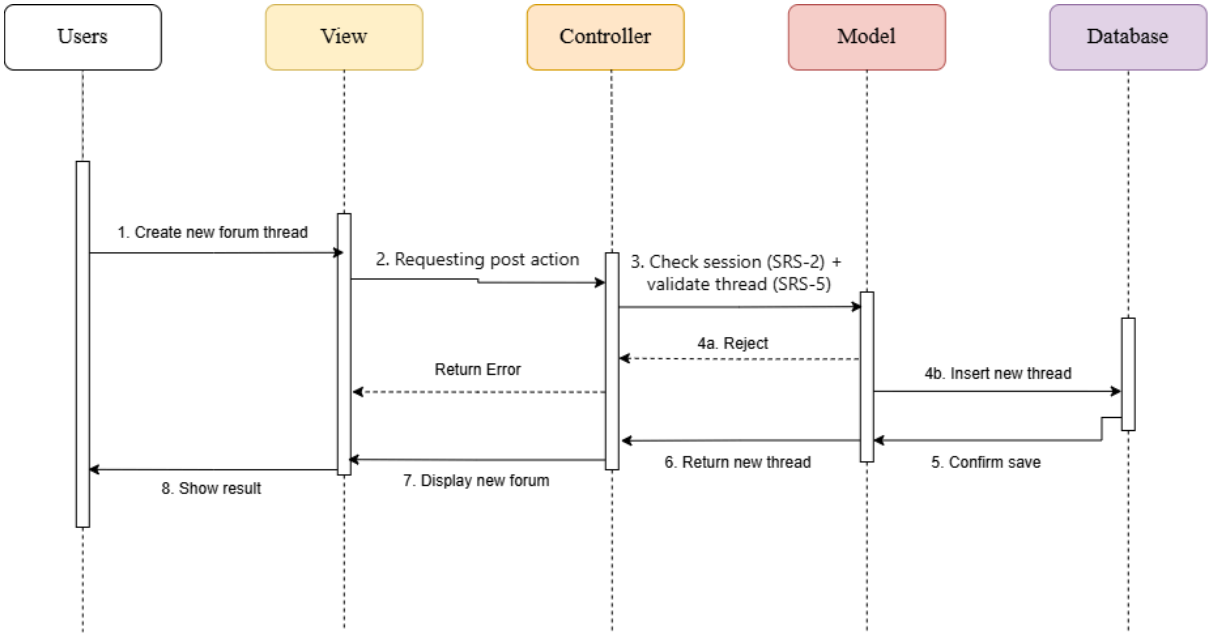
SQD-7: FAQ Browse

Traced: SRS-5 · from AD-2



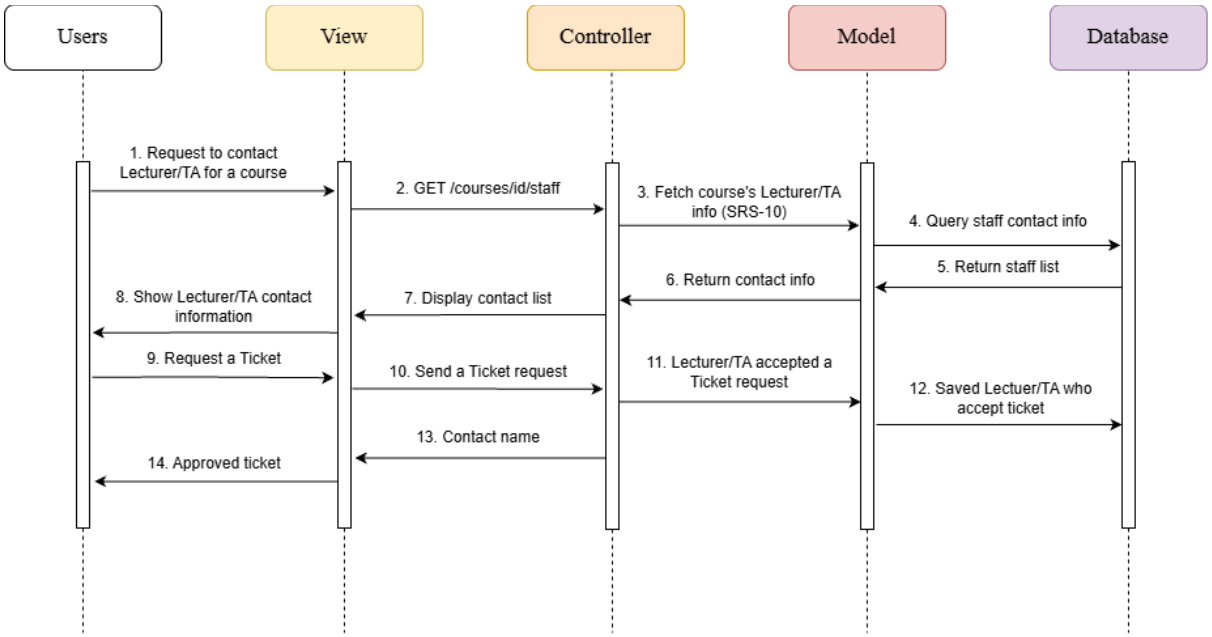
SQD-8: Create New Forum Thread

Traced: SRS-5 · from AD-2



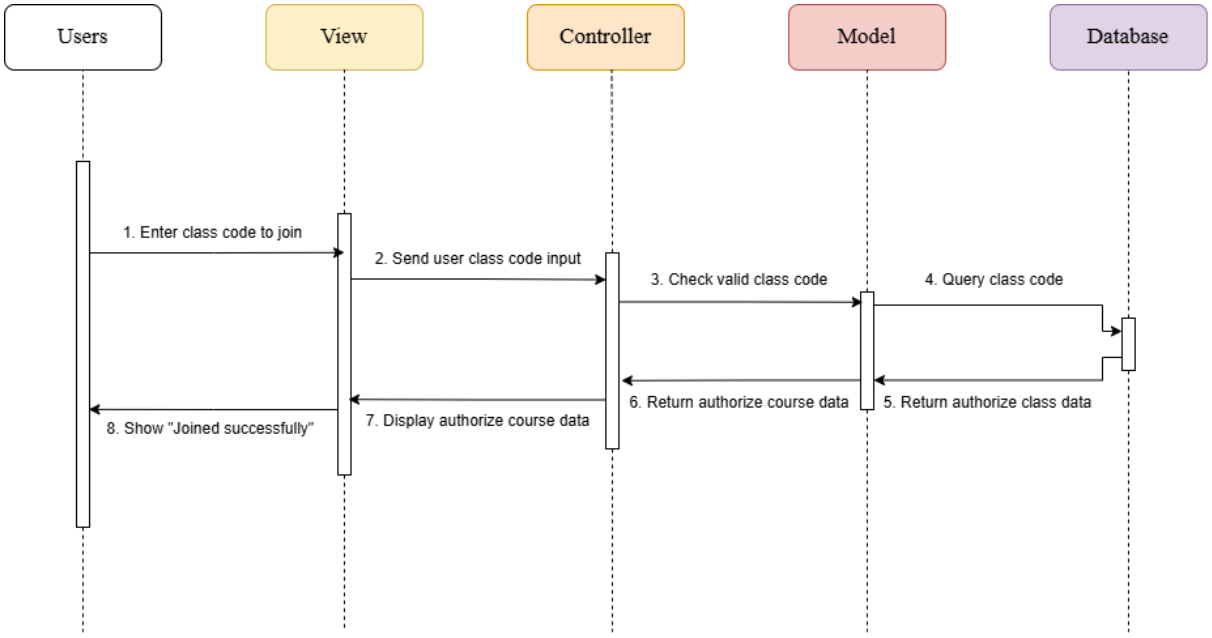
SQD-9: Contact Administrators / Lecturers / TAs

Traced: SRS-10 · from AD-5

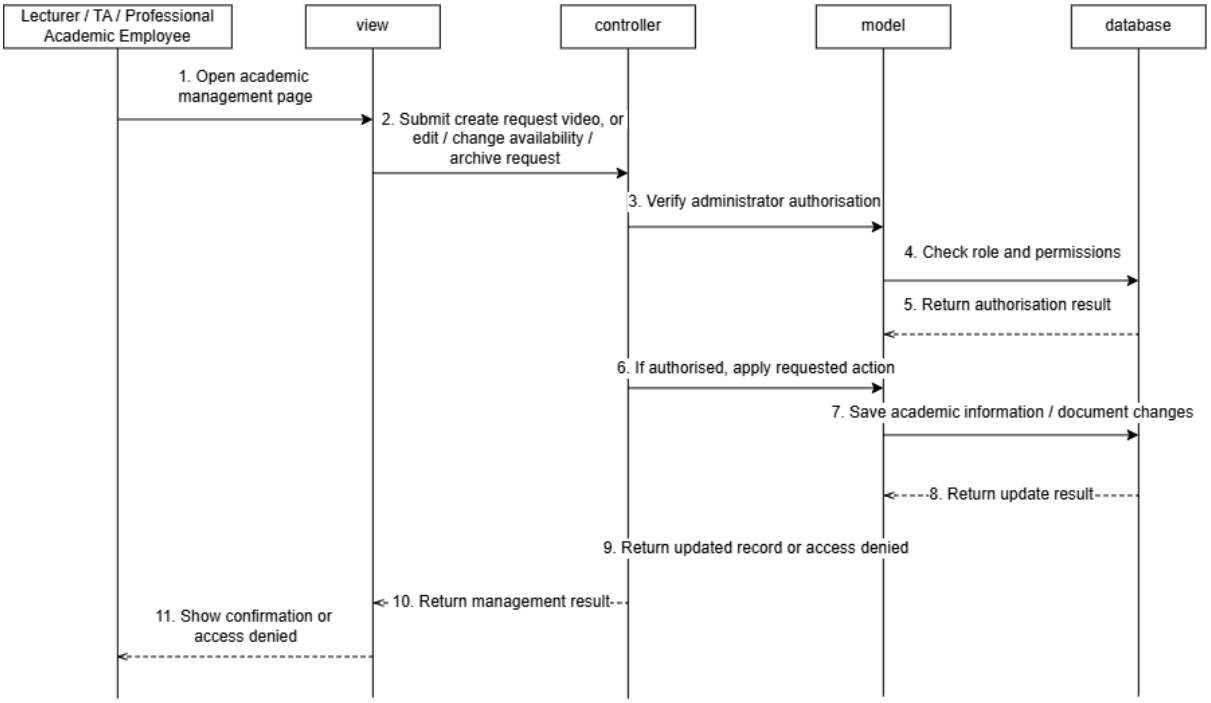


SQD-10: Student Joins Class with Class Code

Traced: SRS-11 · from AD-9



SQD-11: CRUD for Lecturers/TAs about academic material,video,announcements
Traced: SRS-9



13. Traceability Matrix

Sub-Goal	Use Case	User Story	URS	Activity Diagram	SRS
SG-1	View/Search/Ask info about academic & guidance;	US-02	URS-2	AD-2	SRS-3
SG-2	CRUD announcements and information	US-04, US-05	URS-7	AD-1	SRS-9
SG-3	Notifications of announcement; Calendar managing	US-06, US-12	URS-2	AD-4	SRS-4. SRS-6
SG-4	Authentication of roles (SKE, CPE students, etc.)	US-06 US-04, US-12	URS-1 URS-3	AD-8 AD-4	SRS-2 SRS-4, SRS-8
SG-5	Secure authentication via `@ku.th` Google OAuth2	US-01	URS-1	AD-8	SRS-2
SG-6	Searching for information about the lab, announcements and more.	US-03	URS-3	AD-7	SRS-1
SG-7	FAQs (Pantip forum) & Direct Staff Support Channel	US-09, US-10, US-11	URS-4	AD-2 , AD-5	SRS-5 SRS-10
SG-8	Group gathering & Class Code Management	US-13, US-07, US-08	URS-5, URS-6	AD-3 , AD-9	SRS-7 , SRS-11
SG-9	DoomScroll	US-14	URS-9	AD-6	SRS-8

From SRS to Sequence Diagram

SRS	SQD
SRS-1	SQD-1

SRS-2	SQD-1, SQD-2
SRS-3	SQD-1, SQD-3
SRS-4	SQD-6
SRS-5	SQD-7, SQD-8
SRS-6	SQD-6
SRS-7	SQD-4
SRS-8	SQD-5
SRS-9	SQD-11
SRS-10	SQD-9
SRS-11	SQD-10